

## 4.6 Introduction to LR Parsing: Simple LR

The most prevalent type of bottom-up parser today is based on a concept called LR( $k$ ) parsing; the “L” is for left-to-right scanning of the input, the “R” for constructing a rightmost derivation in reverse, and the  $k$  for the number of input symbols of lookahead that are used in making parsing decisions. The cases  $k = 0$  or  $k = 1$  are of practical interest, and we shall only consider LR parsers with  $k \leq 1$  here. When ( $k$ ) is omitted,  $k$  is assumed to be 1.

This section introduces the basic concepts of LR parsing and the easiest method for constructing shift-reduce parsers, called “simple LR” (or SLR, for short). Some familiarity with the basic concepts is helpful even if the LR parser itself is constructed using an automatic parser generator. We begin with “items” and “parser states;” the diagnostic output from an LR parser generator typically includes parser states, which can be used to isolate the sources of parsing conflicts.

Section 4.7 introduces two, more complex methods — canonical-LR and LALR — that are used in the majority of LR parsers.

### 4.6.1 Why LR Parsers?

LR parsers are table-driven, much like the nonrecursive LL parsers of Section 4.4.4. A grammar for which we can construct a parsing table using one of the methods in this section and the next is said to be an *LR grammar*. Intuitively, for a grammar to be LR it is sufficient that a left-to-right shift-reduce parser be able to recognize handles of right-sentential forms when they appear on top of the stack.

LR parsing is attractive for a variety of reasons:

- LR parsers can be constructed to recognize virtually all programming-language constructs for which context-free grammars can be written. Non-LR context-free grammars exist, but these can generally be avoided for typical programming-language constructs.

- The LR-parsing method is the most general nonbacktracking shift-reduce parsing method known, yet it can be implemented as efficiently as other, more primitive shift-reduce methods (see the bibliographic notes).
- An LR parser can detect a syntactic error as soon as it is possible to do so on a left-to-right scan of the input.
- The class of grammars that can be parsed using LR methods is a proper superset of the class of grammars that can be parsed with predictive or LL methods. For a grammar to be  $LR(k)$ , we must be able to recognize the occurrence of the right side of a production in a right-sentential form, with  $k$  input symbols of lookahead. This requirement is far less stringent than that for  $LL(k)$  grammars where we must be able to recognize the use of a production seeing only the first  $k$  symbols of what its right side derives. Thus, it should not be surprising that LR grammars can describe more languages than LL grammars.

The principal drawback of the LR method is that it is too much work to construct an LR parser by hand for a typical programming-language grammar. A specialized tool, an LR parser generator, is needed. Fortunately, many such generators are available, and we shall discuss one of the most commonly used ones, Yacc, in Section 4.9. Such a generator takes a context-free grammar and automatically produces a parser for that grammar. If the grammar contains ambiguities or other constructs that are difficult to parse in a left-to-right scan of the input, then the parser generator locates these constructs and provides detailed diagnostic messages.

### 4.6.2 Items and the $LR(0)$ Automaton

How does a shift-reduce parser know when to shift and when to reduce? For example, with stack contents  $\$T$  and next input symbol  $*$  in Fig. 4.28, how does the parser know that  $T$  on the top of the stack is not a handle, so the appropriate action is to shift and not to reduce  $T$  to  $E$ ?

An LR parser makes shift-reduce decisions by maintaining states to keep track of where we are in a parse. States represent sets of “items.” An  $LR(0)$  item (item for short) of a grammar  $G$  is a production of  $G$  with a dot at some position of the body. Thus, production  $A \rightarrow XYZ$  yields the four items

$$\begin{aligned} A &\rightarrow \cdot XYZ \\ A &\rightarrow X \cdot YZ \\ A &\rightarrow XY \cdot Z \\ A &\rightarrow XYZ \cdot \end{aligned}$$

The production  $A \rightarrow \epsilon$  generates only one item,  $A \rightarrow \cdot$ .

Intuitively, an item indicates how much of a production we have seen at a given point in the parsing process. For example, the item  $A \rightarrow \cdot XYZ$  indicates that we hope to see a string derivable from  $XYZ$  next on the input. Item

### Representing Item Sets

A parser generator that produces a bottom-up parser may need to represent items and sets of items conveniently. Note that an item can be represented by a pair of integers, the first of which is the number of one of the productions of the underlying grammar, and the second of which is the position of the dot. Sets of items can be represented by a list of these pairs. However, as we shall see, the necessary sets of items often include “closure” items, where the dot is at the beginning of the body. These can always be reconstructed from the other items in the set, and we do not have to include them in the list.

$A \rightarrow X \cdot YZ$  indicates that we have just seen on the input a string derivable from  $X$  and that we hope next to see a string derivable from  $YZ$ . Item  $A \rightarrow XYZ \cdot$  indicates that we have seen the body  $XYZ$  and that it may be time to reduce  $XYZ$  to  $A$ .

One collection of sets of LR(0) items, called the *canonical* LR(0) collection, provides the basis for constructing a deterministic finite automaton that is used to make parsing decisions. Such an automaton is called an *LR(0) automaton*.<sup>3</sup> In particular, each state of the LR(0) automaton represents a set of items in the canonical LR(0) collection. The automaton for the expression grammar (4.1), shown in Fig. 4.31, will serve as the running example for discussing the canonical LR(0) collection for a grammar.

To construct the canonical LR(0) collection for a grammar, we define an augmented grammar and two functions, CLOSURE and GOTO. If  $G$  is a grammar with start symbol  $S$ , then  $G'$ , the *augmented grammar* for  $G$ , is  $G$  with a new start symbol  $S'$  and production  $S' \rightarrow S$ . The purpose of this new starting production is to indicate to the parser when it should stop parsing and announce acceptance of the input. That is, acceptance occurs when and only when the parser is about to reduce by  $S' \rightarrow S$ .

### Closure of Item Sets

If  $I$  is a set of items for a grammar  $G$ , then  $\text{CLOSURE}(I)$  is the set of items constructed from  $I$  by the two rules:

1. Initially, add every item in  $I$  to  $\text{CLOSURE}(I)$ .
2. If  $A \rightarrow \alpha \cdot B\beta$  is in  $\text{CLOSURE}(I)$  and  $B \rightarrow \gamma$  is a production, then add the item  $B \rightarrow \cdot \gamma$  to  $\text{CLOSURE}(I)$ , if it is not already there. Apply this rule until no more new items can be added to  $\text{CLOSURE}(I)$ .

<sup>3</sup>Technically, the automaton misses being deterministic according to the definition of Section 3.6.4, because we do not have a dead state, corresponding to the empty set of items. As a result, there are some state-input pairs for which no next state exists.

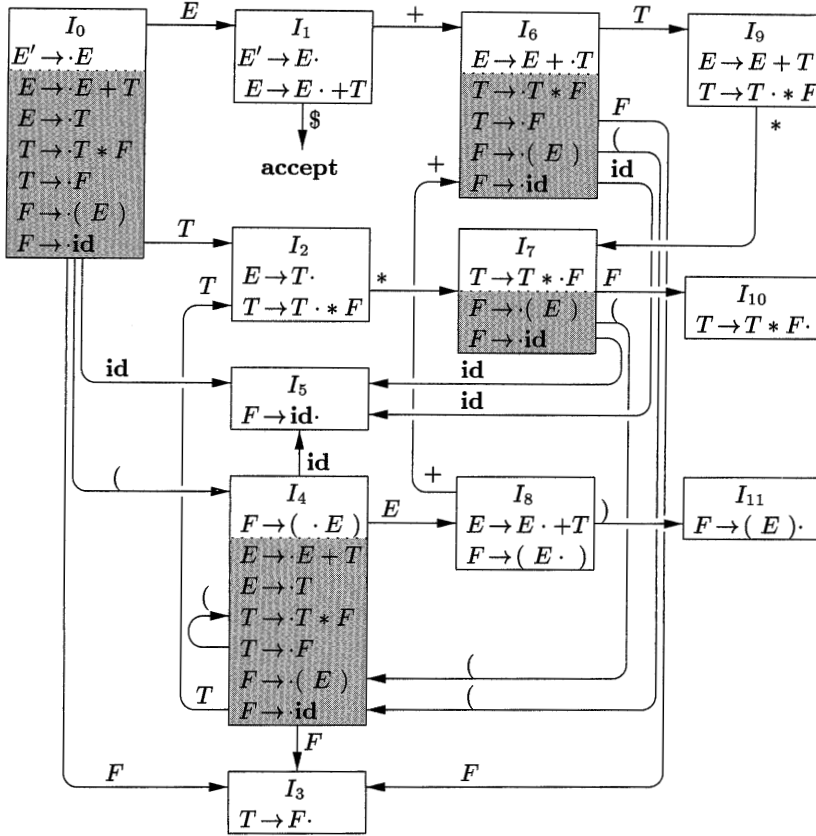


Figure 4.31: LR(0) automaton for the expression grammar (4.1)

Intuitively,  $A \rightarrow \alpha \cdot B \beta$  in  $\text{CLOSURE}(I)$  indicates that, at some point in the parsing process, we think we might next see a substring derivable from  $B\beta$  as input. The substring derivable from  $B\beta$  will have a prefix derivable from  $B$  by applying one of the  $B$ -productions. We therefore add items for all the  $B$ -productions; that is, if  $B \rightarrow \gamma$  is a production, we also include  $B \rightarrow \cdot \gamma$  in  $\text{CLOSURE}(I)$ .

**Example 4.40:** Consider the augmented expression grammar:

$$\begin{aligned}
 E' &\rightarrow E \\
 E &\rightarrow E + T \mid T \\
 T &\rightarrow T * F \mid F \\
 E &\rightarrow (E) \mid \text{id}
 \end{aligned}$$

If  $I$  is the set of one item  $\{[E' \rightarrow \cdot E]\}$ , then  $\text{CLOSURE}(I)$  contains the set of items  $I_0$  in Fig. 4.31.

To see how the closure is computed,  $E' \rightarrow \cdot E$  is put in  $\text{CLOSURE}(I)$  by rule (1). Since there is an  $E$  immediately to the right of a dot, we add the  $E$ -productions with dots at the left ends:  $E \rightarrow \cdot E + T$  and  $E \rightarrow \cdot T$ . Now there is a  $T$  immediately to the right of a dot in the latter item, so we add  $T \rightarrow \cdot T * F$  and  $T \rightarrow \cdot F$ . Next, the  $F$  to the right of a dot forces us to add  $F \rightarrow \cdot (E)$  and  $F \rightarrow \cdot \text{id}$ , but no other items need to be added.  $\square$

The closure can be computed as in Fig. 4.32. A convenient way to implement the function *closure* is to keep a boolean array *added*, indexed by the nonterminals of  $G$ , such that *added*[ $B$ ] is set to **true** if and when we add the item  $B \rightarrow \cdot \gamma$  for each  $B$ -production  $B \rightarrow \gamma$ .

```

SetOfItems CLOSURE( $I$ ) {
     $J = I$ ;
    repeat
        for ( each item  $A \rightarrow \alpha \cdot B \beta$  in  $J$  )
            for ( each production  $B \rightarrow \gamma$  of  $G$  )
                if (  $B \rightarrow \cdot \gamma$  is not in  $J$  )
                    add  $B \rightarrow \cdot \gamma$  to  $J$ ;
    until no more items are added to  $J$  on one round;
    return  $J$ ;
}

```

Figure 4.32: Computation of CLOSURE

Note that if one  $B$ -production is added to the closure of  $I$  with the dot at the left end, then all  $B$ -productions will be similarly added to the closure. Hence, it is not necessary in some circumstances actually to list the items  $B \rightarrow \cdot \gamma$  added to  $I$  by CLOSURE. A list of the nonterminals  $B$  whose productions were so added will suffice. We divide all the sets of items of interest into two classes:

1. *Kernel items*: the initial item,  $S' \rightarrow \cdot S$ , and all items whose dots are not at the left end.
2. *Nonkernel items*: all items with their dots at the left end, except for  $S' \rightarrow \cdot S$ .

Moreover, each set of items of interest is formed by taking the closure of a set of kernel items; the items added in the closure can never be kernel items, of course. Thus, we can represent the sets of items we are really interested in with very little storage if we throw away all nonkernel items, knowing that they could be regenerated by the closure process. In Fig. 4.31, nonkernel items are in the shaded part of the box for a state.

### The Function GOTO

The second useful function is  $\text{GOTO}(I, X)$  where  $I$  is a set of items and  $X$  is a grammar symbol.  $\text{GOTO}(I, X)$  is defined to be the closure of the set of all items  $[A \rightarrow \alpha X \cdot \beta]$  such that  $[A \rightarrow \alpha \cdot X \beta]$  is in  $I$ . Intuitively, the  $\text{GOTO}$  function is used to define the transitions in the LR(0) automaton for a grammar. The states of the automaton correspond to sets of items, and  $\text{GOTO}(I, X)$  specifies the transition from the state for  $I$  under input  $X$ .

**Example 4.41:** If  $I$  is the set of two items  $\{[E' \rightarrow E \cdot], [E \rightarrow E \cdot + T]\}$ , then  $\text{GOTO}(I, +)$  contains the items

$$\begin{aligned} E &\rightarrow E + \cdot T \\ T &\rightarrow \cdot T * F \\ T &\rightarrow \cdot F \\ F &\rightarrow \cdot (E) \\ F &\rightarrow \cdot \text{id} \end{aligned}$$

We computed  $\text{GOTO}(I, +)$  by examining  $I$  for items with  $+$  immediately to the right of the dot.  $E' \rightarrow E \cdot$  is not such an item, but  $E \rightarrow E \cdot + T$  is. We moved the dot over the  $+$  to get  $E \rightarrow E + \cdot T$  and then took the closure of this singleton set.  $\square$

We are now ready for the algorithm to construct  $C$ , the canonical collection of sets of LR(0) items for an augmented grammar  $G'$  — the algorithm is shown in Fig. 4.33.

```

void items( $G'$ ) {
     $C = \text{CLOSURE}(\{[S' \rightarrow \cdot S]\})$ ;
    repeat
        for ( each set of items  $I$  in  $C$  )
            for ( each grammar symbol  $X$  )
                if (  $\text{GOTO}(I, X)$  is not empty and not in  $C$  )
                    add  $\text{GOTO}(I, X)$  to  $C$ ;
    until no new sets of items are added to  $C$  on a round;
}

```

Figure 4.33: Computation of the canonical collection of sets of LR(0) items

**Example 4.42:** The canonical collection of sets of LR(0) items for grammar (4.1) and the  $\text{GOTO}$  function are shown in Fig. 4.31.  $\text{GOTO}$  is encoded by the transitions in the figure.  $\square$

### Use of the LR(0) Automaton

The central idea behind “Simple LR,” or SLR, parsing is the construction from the grammar of the LR(0) automaton. The states of this automaton are the sets of items from the canonical LR(0) collection, and the transitions are given by the GOTO function. The LR(0) automaton for the expression grammar (4.1) appeared earlier in Fig. 4.31.

The start state of the LR(0) automaton is  $\text{CLOSURE}(\{[S' \rightarrow \cdot S]\})$ , where  $S'$  is the start symbol of the augmented grammar. All states are accepting states. We say “state  $j$ ” to refer to the state corresponding to the set of items  $I_j$ .

How can LR(0) automata help with shift-reduce decisions? Shift-reduce decisions can be made as follows. Suppose that the string  $\gamma$  of grammar symbols takes the LR(0) automaton from the start state 0 to some state  $j$ . Then, shift on next input symbol  $a$  if state  $j$  has a transition on  $a$ . Otherwise, we choose to reduce; the items in state  $j$  will tell us which production to use.

The LR-parsing algorithm to be introduced in Section 4.6.3 uses its stack to keep track of states as well as grammar symbols; in fact, the grammar symbol can be recovered from the state, so the stack holds states. The next example gives a preview of how an LR(0) automaton and a stack of states can be used to make shift-reduce parsing decisions.

**Example 4.43:** Figure 4.34 illustrates the actions of a shift-reduce parser on input **id \* id**, using the LR(0) automaton in Fig. 4.31. We use a stack to hold states; for clarity, the grammar symbols corresponding to the states on the stack appear in column SYMBOLS. At line (1), the stack holds the start state 0 of the automaton; the corresponding symbol is the bottom-of-stack marker \$.

| LINE | STACK    | SYMBOLS              | INPUT                 | ACTION                                |
|------|----------|----------------------|-----------------------|---------------------------------------|
| (1)  | 0        | \$                   | <b>id * id</b> \$     | shift to 5                            |
| (2)  | 0 5      | \$ <b>id</b>         | <b>*</b> <b>id</b> \$ | reduce by $F \rightarrow \mathbf{id}$ |
| (3)  | 0 3      | \$ $F$               | <b>*</b> <b>id</b> \$ | reduce by $T \rightarrow F$           |
| (4)  | 0 2      | \$ $T$               | <b>*</b> <b>id</b> \$ | shift to 7                            |
| (5)  | 0 2 7    | \$ $T *$             | <b>id</b> \$          | shift to 5                            |
| (6)  | 0 2 7 5  | \$ $T * \mathbf{id}$ | \$                    | reduce by $F \rightarrow \mathbf{id}$ |
| (7)  | 0 2 7 10 | \$ $T * F$           | \$                    | reduce by $T \rightarrow T * F$       |
| (8)  | 0 2      | \$ $T$               | \$                    | reduce by $E \rightarrow T$           |
| (9)  | 0 1      | \$ $E$               | \$                    | accept                                |

Figure 4.34: The parse of **id \* id**

The next input symbol is **id** and state 0 has a transition on **id** to state 5. We therefore shift. At line (2), state 5 (symbol **id**) has been pushed onto the stack. There is no transition from state 5 on input **\***, so we reduce. From item  $[F \rightarrow \mathbf{id} \cdot]$  in state 5, the reduction is by production  $F \rightarrow \mathbf{id}$ .

With symbols, a reduction is implemented by popping the body of the production from the stack (on line (2), the body is **id**) and pushing the head of the production (in this case,  $F$ ). With states, we pop state 5 for symbol **id**, which brings state 0 to the top and look for a transition on  $F$ , the head of the production. In Fig. 4.31, state 0 has a transition on  $F$  to state 3, so we push state 3, with corresponding symbol  $F$ ; see line (3).

As another example, consider line (5), with state 7 (symbol  $*$ ) on top of the stack. This state has a transition to state 5 on input **id**, so we push state 5 (symbol **id**). State 5 has no transitions, so we reduce by  $F \rightarrow \mathbf{id}$ . When we pop state 5 for the body **id**, state 7 comes to the top of the stack. Since state 7 has a transition on  $F$  to state 10, we push state 10 (symbol  $F$ ).  $\square$